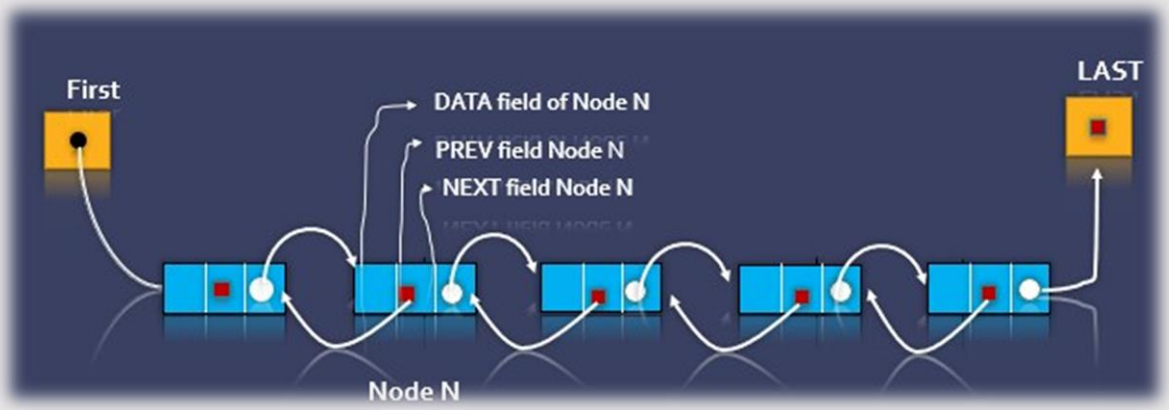
Lab#03
Double Linked List



Prepared by: Engr. Amna Arooj

FCSE, GIKI

Lab 03
Doubly Linked List

3.1 Doubly Linked List

A doubly linked list is a type of linked list in which each node consists of 3 components:

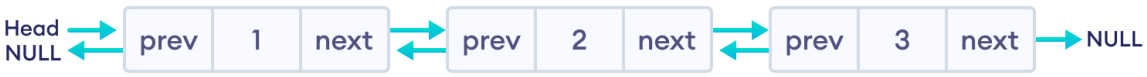
- `*prev` - address of the previous node
- `data` - data item
- `*next` - address of next node



A doubly linked list node.

3.2 Representation of Doubly Linked List

Let's see how we can represent a doubly linked list on an algorithm/code. Suppose we have a doubly linked list:



Newly created doubly linked list

Here, the single node is represented as

```
struct node {
    int data;
    struct node *next;
    struct node *prev;
}
```

Each struct node has a data item, a pointer to the previous struct node, and a pointer to the next struct node.

Now we will create a simple doubly linked list with three items to understand how this works.

```
/* Initialize nodes */
struct node *head;
struct node *one = NULL;
struct node *two = NULL;
struct node *three = NULL;

/* Allocate memory */
one = malloc(sizeof(struct node));
two = malloc(sizeof(struct node));
three = malloc(sizeof(struct node));

/* Assign data values */
one->data = 1;
two->data = 2;
three->data = 3;

/* Connect nodes */
one->next = two;
one->prev = NULL;

two->next = three;
two->prev = one;

three->next = NULL;
three->prev = two;

/* Save address of first node in head */
head = one;
```

In the above code, `one`, `two`, and `three` are the nodes with data items **1**, **2**, and **3** respectively.

For node one: `next` stores the address of `two` and `prev` stores `null` (there is no node before it)

For node two: `next` stores the address of `three` and `prev` stores the address of `one`

For node three: `next` stores `null` (there is no node after it) and `prev` stores the address of `two`.

Note: In the case of the head node, `prev` points to `null`, and in the case of the tail pointer, `next` points to null. Here, `one` is a head node and `three` is a tail node.

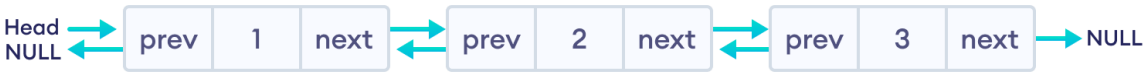
3.3 Insertion on a Doubly Linked List

Pushing a node to a doubly-linked list is similar to pushing a node to a linked list, but extra work is required to handle the pointer to the previous node.

We can insert elements at 3 different positions of a doubly-linked list:

- Insertion at the beginning
- Insertion in-between nodes
- Insertion at the End

Suppose we have a double-linked list with elements **1**, **2**, and **3**.



Original doubly linked list

3.3.1 Insertion at the Beginning

Let's add a node with value **6** at the beginning of the doubly linked list we made above.

1. Create a new node

allocate memory for `newNode`

assign the data to `newNode`.

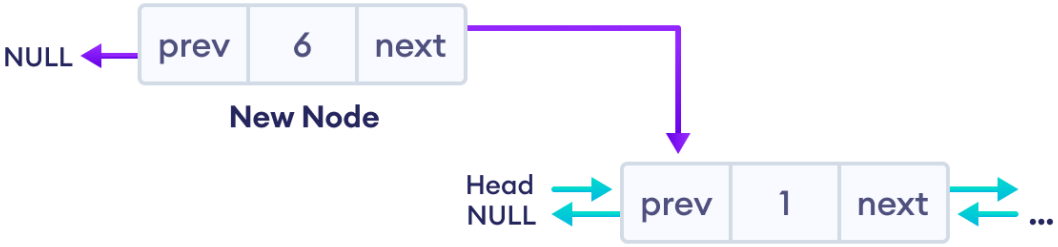


New node

2. Set prev and next pointers of new node

point `next` of `newNode` to the first node of the doubly linked list

point `prev` to `null`

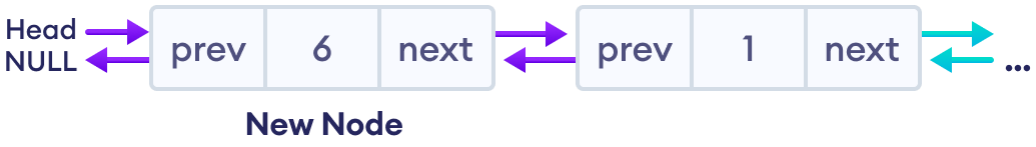


Reorganize the pointers (changes are denoted by purple arrows)

3. Make new node as head node

Point `prev` of the first node to `newNode` (now the previous `head` is the second node)

Point `head` to `newNode`



Reorganize the pointers

Code for Insertion at the Beginning

```
// insert node at the front
void insertFront(struct Node** head, int data) {


   // allocate memory for newNode
```

```
    struct Node* newNode = new Node;

    // assign data to newNode
    newNode->data = data;

    // point next of newNode to the first node of the doubly linked list
    newNode->next = (*head);

    // point prev to NULL
    newNode->prev = NULL;

    // point previous of the first node (now first node is the second node) to newNode
    if ((*head) != NULL)
        (*head)->prev = newNode;

    // head points to newNode
    (*head) = newNode;
}
```

3.3.2 Insertion in between two nodes

Let's add a node with value 6 after node with value 1 in the doubly linked list.

1. Create a new node

allocate memory for `newNode`

assign the data to `newNode`.

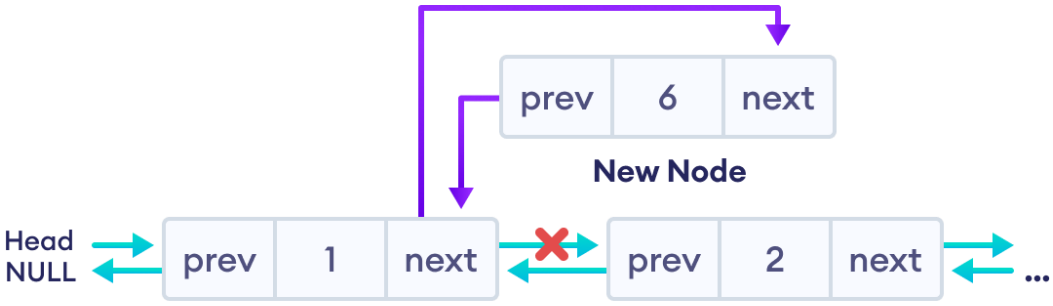


New node

2. Set the next pointer of new node and previous node

assign the value of `next` from previous node to the `next` of `newNode`

assign the address of `newNode` to the `next` of previous node

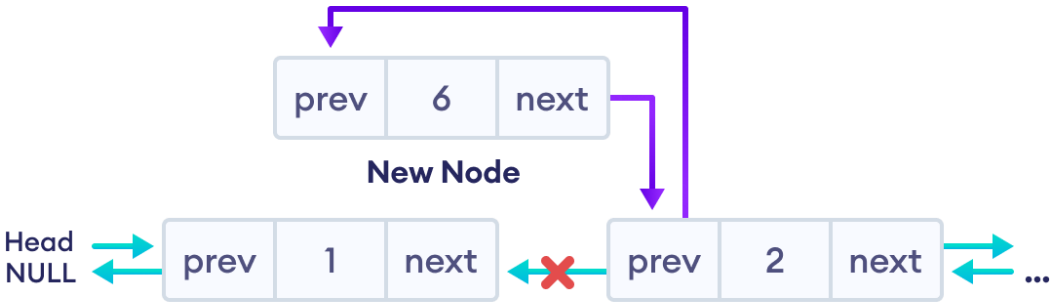


Reorganize the pointers

3. Set the prev pointer of new node and the next node

assign the value of `prev` of next node to the `prev` of `newNode`

assign the address of `newNode` to the `prev` of next node



Reorganize the pointers

The final doubly linked list is after this insertion is:



Final list

Code for Insertion in between two Nodes

```
// insert a node after a specific node
```

```
void insertAfter(struct Node* prev_node, int data) {

    // check if previous node is NULL
    if (prev_node == NULL) {
        cout << "previous node cannot be NULL";
        return;
    }

    // allocate memory for newNode
    struct Node* newNode = new Node;

    // assign data to newNode
    newNode->data = data;

    // set next of newNode to next of prev node
    newNode->next = prev_node->next;

    // set next of prev node to newNode
    prev_node->next = newNode;

    // set prev of newNode to the previous node
    newNode->prev = prev_node;

    // set prev of newNode's next to newNode
    if (newNode->next != NULL)
        newNode->next->prev = newNode;
}
```

3.3.3. Insertion at the End

Let's add a node with value 6 at the end of the doubly linked list.

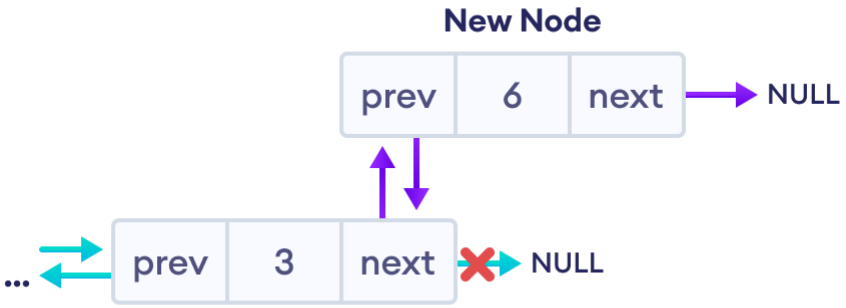
1. Create a new node



New node

2. Set prev and next pointers of new node and the previous node

If the linked list is empty, make the `newNode` as the head node. Otherwise, traverse to the end of the doubly linked list and



Reorganize the pointers

The final doubly linked list looks like this.



The final list

Code for Insertion at the End

```
// insert a newNode at the end of the list

void insertEnd(struct Node** head, int data) {
```

```
// allocate memory for node
struct Node* newNode = new Node;

// assign data to newNode
newNode->data = data;

// assign NULL to next of newNode
newNode->next = NULL;

// store the head node temporarily (for later use)
struct Node* temp = *head;

// if the linked list is empty, make the newNode as head node
if (*head == NULL) {
    newNode->prev = NULL;
    *head = newNode;
    return;
}

// if the linked list is not empty, traverse to the end of the linked list
while (temp->next != NULL)
    temp = temp->next;

// now, the last node of the linked list is temp

// point the next of the last node (temp) to newNode.
temp->next = newNode;

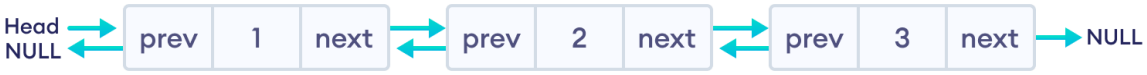
// assign prev of newNode to temp
newNode->prev = temp;
```

```
}
```

3.4 Deletion from a Doubly Linked List

Similar to insertion, we can also delete a node from **3** different positions of a doubly linked list.

Suppose we have a double-linked list with elements **1**, **2**, and **3**.

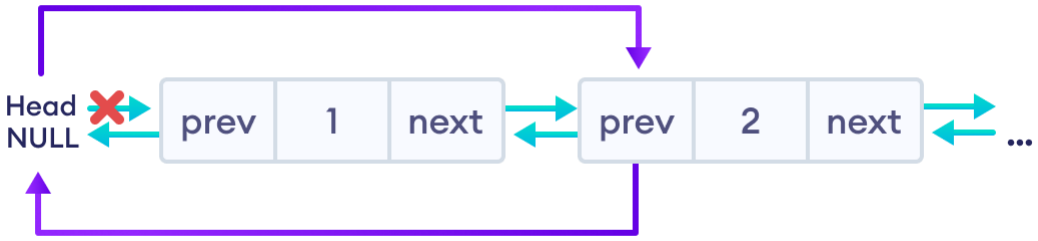


Original doubly linked list

3.4.1 Delete the First Node of Doubly Linked List

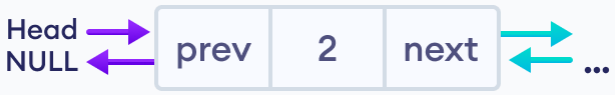
If the node to be deleted (i.e. `del_node`) is at the beginning

Reset value node after the del_node (i.e. node two)



Reorganize the pointers

Finally, free the memory of `del_node`. And, the linked will look like this



Free the space of the first node

Final list

Code for Deletion of the First Node

```
if (*head == del_node)
    *head = del_node->next;

if (del_node->prev != NULL)
    del_node->prev->next = del_node->next;

free(del);
```

3.4.2 Deletion of the Inner Node

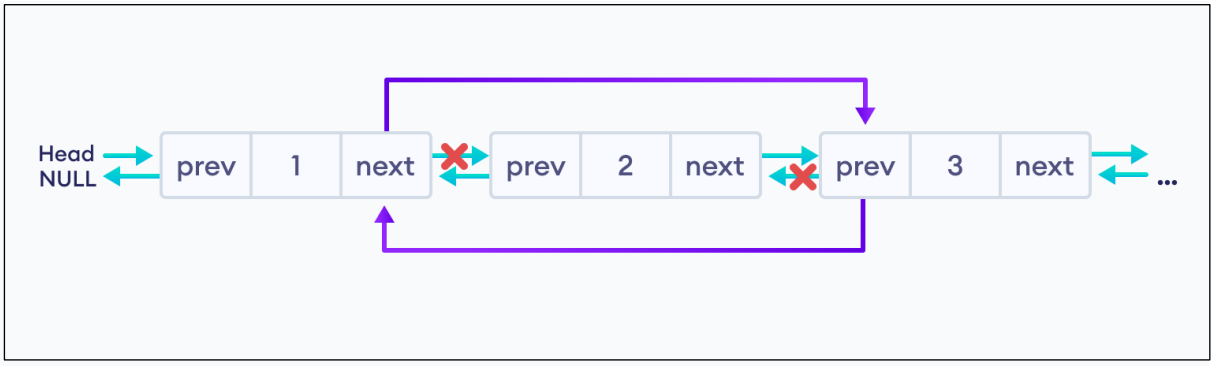
If `del_node` is an inner node (second node), we must have to reset the value of `next` and `prev` of the nodes before and after the `del_node`.

For the node before the del_node (i.e. first node)

Assign the value of `next` of `del_node` to the `next` of the `first` node.

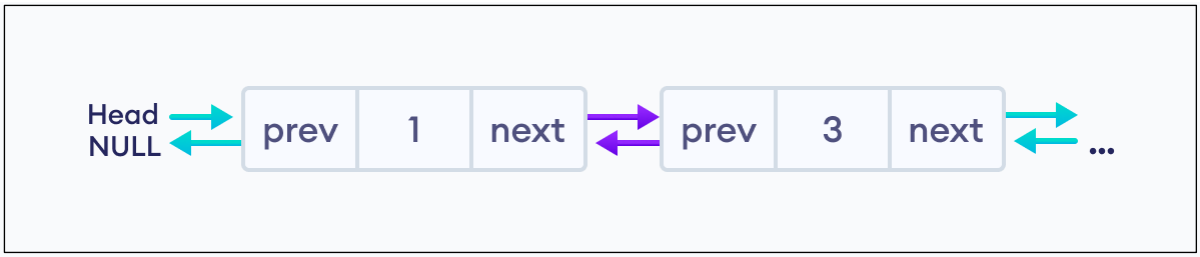
For the node after the del_node (i.e. third node)

Assign the value of `prev` of `del_node` to the `prev` of the `third` node.



Reorganize the pointers

Finally, we will free the memory of `del_node`. And, the final doubly linked list looks like this.



Final list

Code for Deletion of the Inner Node

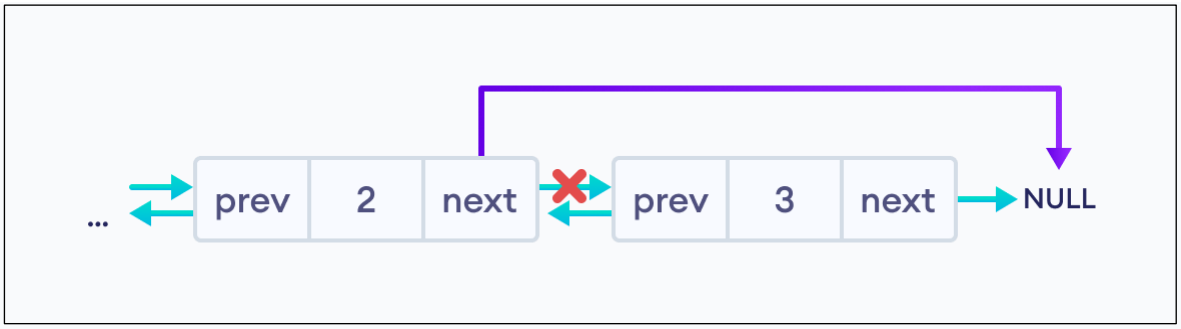
```
if (del_node->next != NULL)
    del_node->next->prev = del_node->prev;


if (del_node->prev != NULL)
    del_node->prev->next = del_node->next;
```

3.3.3 Delete the Last Node of Doubly Linked List

In this case, we are deleting the last node with value **3** of the doubly linked list.

Here, we can simply delete the `del_node` and make the `next` of node before `del_node` point to `NULL`.



Reorganize the pointers

The final doubly linked list looks like this.



Final list

Code for Deletion of the Last Node

```
if (del_node->prev != NULL)
    del_node->prev->next = del_node->next;
```

Here, `del_node ->next` is `NULL` so `del_node->prev->next = NULL`.

Note: We can also solve this using the first condition (for the node before `del_node`) of the second case (Delete the inner node).

3.4 Doubly Linked List Code in C++

```
#include <iostream>
using namespace std;

// node creation
struct Node {
  int data;
  struct Node* next;
  struct Node* prev;
};

// insert node at the front
void insertFront(struct Node** head, int data) {
  // allocate memory for newNode
  struct Node* newNode = new Node;

  // assign data to newNode
  newNode->data = data;

  // make newNode as a head
  newNode->next = (*head);

  // assign null to prev
  newNode->prev = NULL;
```

```
  // previous of head (now head is the second node) is newNode
  if ((*head) != NULL)
    (*head)->prev = newNode;

  // head points to newNode
  (*head) = newNode;
}

// insert a node after a specific node
void insertAfter(struct Node* prev_node, int data) {
  // check if previous node is null
  if (prev_node == NULL) {
    cout << "previous node cannot be null";
    return;
  }

  // allocate memory for newNode
  struct Node* newNode = new Node;

  // assign data to newNode
  newNode->data = data;

  // set next of newNode to next of prev node
  newNode->next = prev_node->next;

  // set next of prev node to newNode
  prev_node->next = newNode;

  // set prev of newNode to the previous node
```

```
  newNode->prev = prev_node;

  // set prev of newNode's next to newNode
  if (newNode->next != NULL)
    newNode->next->prev = newNode;
}

// insert a newNode at the end of the list
void insertEnd(struct Node** head, int data) {
  // allocate memory for node
  struct Node* newNode = new Node;

  // assign data to newNode
  newNode->data = data;

  // assign null to next of newNode
  newNode->next = NULL;

  // store the head node temporarily (for later use)
  struct Node* temp = *head;

  // if the linked list is empty, make the newNode as head node
  if (*head == NULL) {
    newNode->prev = NULL;
    *head = newNode;
    return;
  }

  // if the linked list is not empty, traverse to the end of the linked list
  while (temp->next != NULL)
```

```
    temp = temp->next;

  // now, the last node of the linked list is temp

  // assign next of the last node (temp) to newNode
  temp->next = newNode;

  // assign prev of newNode to temp
  newNode->prev = temp;
}

// delete a node from the doubly linked list
void deleteNode(struct Node** head, struct Node* del_node) {
  // if head or del is null, deletion is not possible
  if (*head == NULL || del_node == NULL)
    return;

  // if del_node is the head node, point the head pointer to the next of del_node
  if (*head == del_node)
    *head = del_node->next;

  // if del_node is not at the last node, point the prev of node next to del_node to the previous
of del_node
  if (del_node->next != NULL)
    del_node->next->prev = del_node->prev;

  // if del_node is not the first node, point the next of the previous node to the next node of
del_node
  if (del_node->prev != NULL)
    del_node->prev->next = del_node->next;
```

```
  // free the memory of del_node
  free(del_node);
}

// print the doubly linked list
void displayList(struct Node* node) {
  struct Node* last;

  while (node != NULL) {
    cout << node->data << "->";
    last = node;
    node = node->next;
  }
  if (node == NULL)
    cout << "NULL\n";
}

int main() {
  // initialize an empty node
  struct Node* head = NULL;

  insertEnd(&head, 5);
  insertFront(&head, 1);
  insertFront(&head, 6);
  insertEnd(&head, 9);

  // insert 11 after head
  insertAfter(head, 11);

  // insert 15 after the seond node
```

```
 insertAfter(head->next, 15);


 displayList(head);


 // delete the last node
 deleteNode(&head, head->next->next->next->next->next);


 displayList(head);
}
```

3.5 Doubly Linked List Complexity

Doubly Linked List Complexity	Time Complexity	Space Complexity
Insertion Operation	O(1) or O(n)	O(1)
Deletion Operation	O(1)	O(1)

1. Complexity of Insertion Operation

The insertion operations that do not require traversal have the time complexity of `O(1)`.

And, insertion that requires traversal has time complexity of `O(n)`.

The space complexity is `O(1)`.

2. Complexity of Deletion Operation

All deletion operations run with time complexity of `O(1)`.

And, the space complexity is `O(1)`.

3.6 Doubly Linked List Applications

Redo and undo functionality in software.

Forward and backward navigation in browsers.

For navigation systems where forward and backward navigation is required.

3.7 Singly Linked List Vs Doubly Linked List

Singly Linked List	Doubly Linked List
Each node consists of a data value and a pointer to the next node.	Each node consists of a data value, a pointer to the next node, and a pointer to the previous node.
Traversal can occur in one way only (forward direction).	Traversal can occur in both ways.
It requires less space.	It requires more space because of an extra pointer.
It can be implemented on the stack.	It has multiple usages. It can be implemented on the stack, heap, and binary tree.

3.8 Practice Problems

Exercise 1

Design doubly linked list having basic functionality of adding an element at the end of the list and display all the entered list elements when user entered -1. Also implement the functionality for deletion in doubly linked list based upon the data (value in the list).

Exercise 2

Write a C++ program to check if a double linked list is a palindrome or not.
Test Data and Expected Output :

```
Original List:
1 2 3 4 5
Linked list is not a palindrome.

Original Singly List:
1 2 2 1
Linked list is a palindrome.

Original List:
```

```
MADAM
Linked list is a palindrome.
```

Exercise 3

Write a C++ program to remove duplicates from a double unsorted linked list.
Test Data and Expected Output :

```
Original List:
1 2 3 3 4
After removing duplicate elements from the said singly list:
1 2 3 4

Original List:
1 2 3 3 4 4
After removing duplicate elements from the said singly list:
1 2 3 4
```